Guía de trabajo en equipo con Git Flow

¿Qué es Git Flow?

Git Flow es un modelo de ramificación creado por Vincent Driessen. Define exactamente qué ramas existen y para qué sirve cada una, permitiendo que un equipo trabaje de forma ordenada y sin pisarse el código.

Las ramas principales

Rama	Propósito
`main`	Código en producción. Siempre estable.
`develop`	Base de desarrollo. Todos integran aquí.
`feature/xxx`	Nueva funcionalidad.
`release/xxx`	Preparación antes de subir a producción.
`hotfix/xxx`	Corrección urgente de un bug en producción.

Diagrama del flujo





Detalle de cada rama

`feature/`

- **Se crea desde:** `develop`
- **Se mergea a:** `develop`
- **Para qué:** Desarrollar una nueva funcionalidad
- **Regla:** Una rama por funcionalidad, no mezclar cosas

```
git checkout develop
git pull origin develop
git checkout -b feature/auth-login
# ... desarrollas ...
git push origin feature/auth-login
# Crear Pull Request hacia develop en GitHub
```

`release/`

- **Se crea desde:** `develop`
- **Se mergea a:** `main` y `develop`
- **Para qué:** Preparar una versión para producción sin detener el desarrollo

Cuando `develop` tiene suficientes features listas, se crea una `release` para:

1. Hacer pruebas finales
2. Corregir bugs menores encontrados
3. Actualizar versión en `package.json` y documentación

> En `release` NO se agregan features nuevos, solo correcciones.

```
git checkout develop
git checkout -b release/1.0
# pruebas, correcciones menores...
git commit -m "fix: corregir validación de DNI"
# merge a main (producción) y a develop (para conservar l
```

`hotfix/`

- **Se crea desde:** `main`
- **Se mergea a:** `main` y `develop`
- **Para qué:** Corregir un bug urgente que está en producción

```
git checkout main
git checkout -b hotfix/fix-login-crash
# corriges el bug...
git commit -m "fix: corregir crash en login"
# merge a main y develop
```

Flujo de trabajo diario

```
# 1. Siempre partir desde develop actualizado
git checkout develop
git pull origin develop

# 2. Crear tu rama para la tarea
git checkout -b feature/nombre-funcionalidad
```

```
# 3. Trabajar y hacer commits pequeños y frecuentes
git add .
git commit -m "feat: descripción del cambio"

# 4. Antes de subir, sincronizar con develop
git pull origin develop

# 5. Subir tu rama
git push origin feature/nombre-funcionalidad

# 6. Crear Pull Request hacia develop en GitHub
# 7. Un compañero revisa y aprueba
# 8. Se hace merge a develop
```

Convención de commits

Usar mensajes de commit claros y consistentes:

Prefijo	Uso
`feat:`	Nueva funcionalidad
`fix:`	Corrección de bug
`refactor:`	Refactorización de código
`docs:`	Cambios en documentación
`chore:`	Configuración, dependencias
`test:`	Pruebas

Ejemplos:

```
feat: agregar endpoint de login
fix: corregir validación de DNI duplicado
refactor: separar lógica de personas en caso de uso
docs: actualizar README con instrucciones de despliegue
chore: migrar de npm a pnpm
```

Reglas de oro para el equipo

1. **Nunca pushear directo a `main` ni `develop`**, siempre mediante Pull Request
2. **Un PR = una funcionalidad**, no mezclar tareas en la misma rama
3. **Hacer pull de develop antes de crear una rama nueva**
4. **Nunca subir `.env`, `dist`, `node_modules`** al repositorio
5. **Todo PR debe ser revisado por al menos un compañero** antes de mergear
6. **Resolver conflictos en tu rama**, no en develop
7. **Cada microservicio tiene su propio repositorio**, coordinar quién trabaja en cuál

Resumen visual

```
DESARROLLO NORMAL:
develop → feature/xxx → (PR) → develop

LANZAMIENTO:
develop → release/x.x → (pruebas) → main + develop

BUG URGENTE EN PRODUCCIÓN:
main → hotfix/xxx → (fix) → main + develop
```